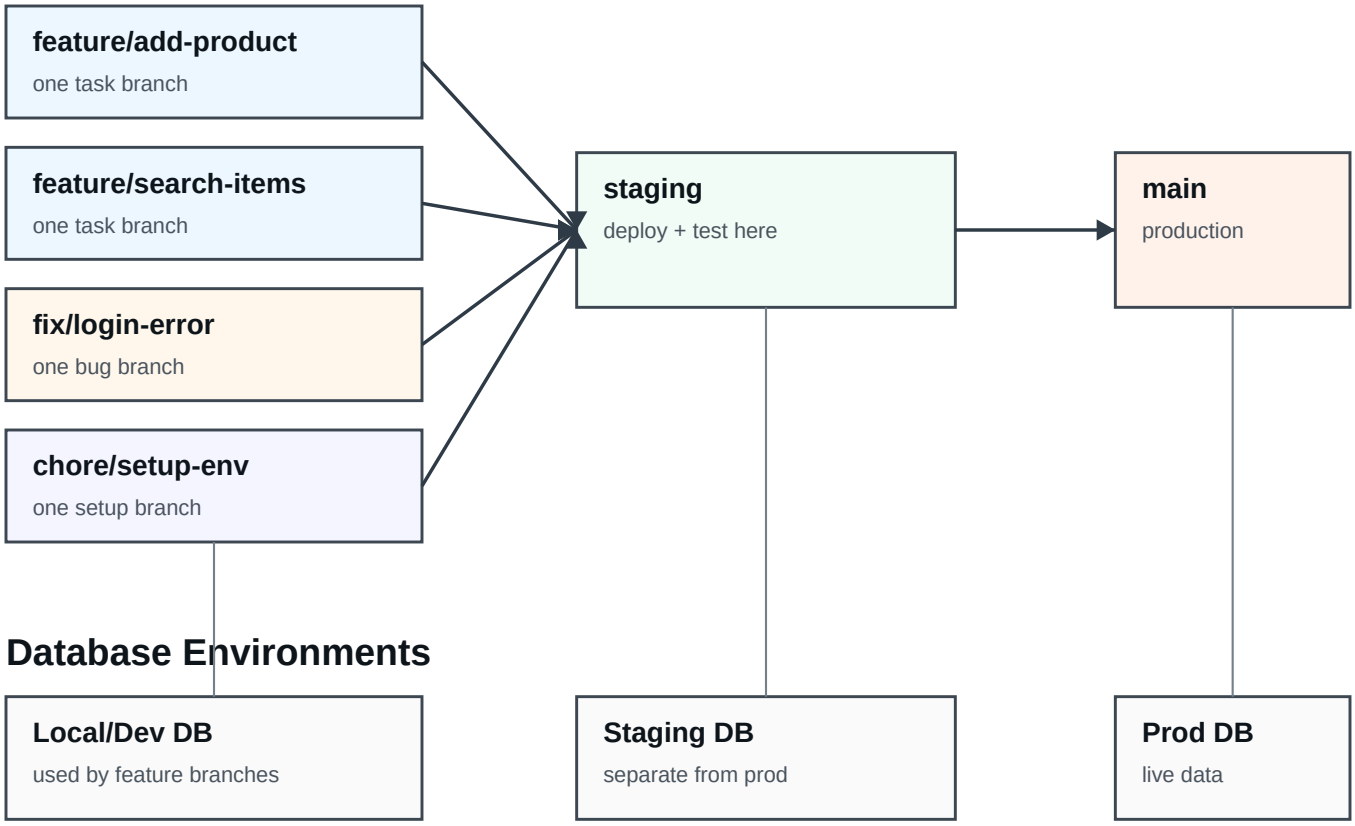
Branching Strategy with Staging

Best practice: create one short-lived branch for each task, feature, bug fix, or chore.

Visual Process



Database Environments

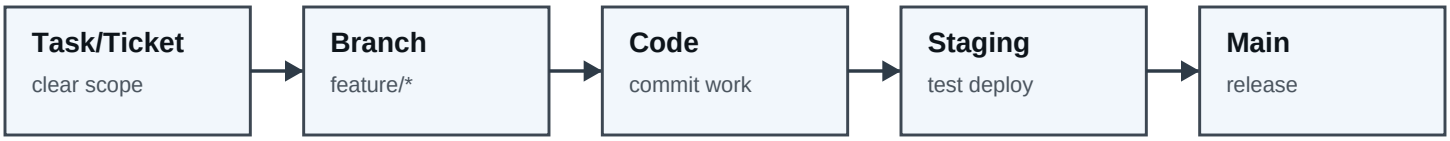
Simple Rule

- Do the coding in feature/*, fix/*, or chore/* branches.
- Merge completed branches into staging first, then test with the staging database.
- Merge staging into main only when it is ready for production.
- Never connect staging to the production database.

How to Work Each Task

Treat every task branch like a small package of work that can be reviewed, tested, and merged.

Task Flow



Commands

```
git checkout staging
git pull origin staging
git checkout -b feature/add-inventory-search
# code, test, commit, then merge into staging
```

Branch Naming

```
feature/add-product-form
feature/inventory-search
feature/create-order-page
fix/login-validation-error
chore/setup-staging-database
```

Best Practice Checklist

- One branch per meaningful task, feature, bug fix, or database change.
- Keep branches small and short-lived.
- Start feature branches from staging so they are based on the latest pre-production code.
- Avoid direct coding on main and avoid normal feature work directly on staging.
- Run migrations on staging before production.
- Delete merged feature branches after they are no longer needed.

Recommended Setup

- main = production app + production database
- staging = staging app + staging database
- feature/*, fix/*, chore/* = daily work + local/dev database

Main idea: your colleague is right. Branch out every single meaningful feature or task.